



SIMATS
ENGINEERING



SIMATS
Saveetha Institute of Medical And Technical Sciences
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

Smart Electric Vehicle Charging Network Management Platform

CI/CD Pipeline Design Document

EVCP Booking & Station Service

<https://github.com/RAKS-rak57/evcp-booking-service->

Submitted in the partial fulfilment for the Course of

CSA1016 – SOFTWARE ENGINEERING

to the award of the degree of

**BACHELOR OF ENGINEERING IN
ARTIFICIAL INTELLIGENCE AND DATA SCIENCE**

Submitted by

R. R. RAKSHITH (192324156)

Under the Supervision of

Dr. K. Anitha Davamani

SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences

Chennai-602105

JUL 2026



SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences
Chennai-602105



DECLARATION

I, R. R. Rakshith, of the Department of Artificial Intelligence & Data Science, Saveetha Institute of Medical and Technical Sciences, Saveetha University, Chennai, hereby declare that the Case study work entitled **Smart Electric Vehicle Charging Network Management Platform** is the result of our own Bonafide efforts. To the best of our knowledge, the work presented herein is original, accurate, and has been carried out in accordance with principles of engineering ethics.

Place: Chennai

Date: 15/08/2026

Signature of the Students with Names

R R RAKSHITH (192324156)

ACKNOWLEDGEMENT

We would like to express our heartfelt gratitude to all those who supported and guided us throughout the successful completion of our Capstone Project. We are deeply thankful to our respected Founder and Chancellor, Dr. N.M. Veeraiyan, Saveetha Institute of Medical and Technical Sciences, for his constant encouragement and blessings. We also express our sincere thanks to our Pro-Chancellor, Dr. Deepak Nallaswamy Veeraiyan, and our Vice-Chancellor, Dr. S. Suresh Kumar, for their visionary leadership and moral support during the course of this project.

We are truly grateful to our Director, Dr. Ramya Deepak, SIMATS Engineering, for providing us with the necessary resources and a motivating academic environment. Our special thanks to our Principal, Dr. B. Ramesh for granting us access to the institute's facilities and encouraging us throughout the process. We sincerely thank our Head of the Department, **Dr. Anusuya** for her continuous support, valuable guidance, and constant motivation.

We are especially indebted to our guide, **Dr. K. Anitha Davamani** for her creative suggestions, consistent feedback, and unwavering support during each stage of the project. We also express our gratitude to the Project Coordinators, Review Panel Members (Internal and External), and the entire faculty team for their constructive feedback and valuable inputs that helped improve the quality of our work. Finally, we thank all faculty members, lab technicians, our parents, and friends for their continuous encouragement and support.

Signature With Student Name

R R RAKSHITH (192324156)

TABLE OF CONTENTS

S.NO	TOPIC	PAGE NO.
1	REQUIREMENT ANALYSIS AND CI/CD NEEDS	3
2	PIPELINE ARCHITECTURE AND WORKFLOW	4
3	TOOLS AND TECHNOLOGY SELECTION	5
4	AUTOMATED TESTING STRATEGY	6
5	DEPLOYMENT STRATEGY	7
6	SECURITY, QUALITY, NOTIFICATIONS AND ROLLBACK	8
7	REAL PIPELINE EXECUTION EVIDENCE	9-13
8	CONCLUSION	14

1. Requirement Analysis and CI/CD Needs

This document designs and implements a real, working CI/CD pipeline for the EVCP Booking & Station Service — the Node.js/Express microservice built in CO3-AT1, code-reviewed in CO3-AT2, and test-designed in CO4-AT1. The pipeline described here is not a theoretical design only: it is implemented as an actual GitHub Actions workflow (.github/workflows/ci-cd.yml) committed to the project's real Git repository, and Section 7 presents genuine execution evidence from a live run.

1.1 Why This Project Needs CI/CD

- Frequent integration: the CO3-AT1 Git history already shows multiple parallel feature branches (booking-api, request-logging, rate-limiting, docker-support) merging into develop — without automation, each merge risks silently breaking the build.
- Regression risk: CO4-AT1's test case design work found 9 genuine defects through manual execution; an automated pipeline ensures the existing test suite runs on every change, not only when a developer remembers to run it.
- Containerized deployment: the service is already Dockerized (CO3-AT1); a pipeline is needed to reliably build, scan, and publish that image rather than relying on manual docker build/push commands.
- Multiple environments: the project's architecture design (CO2-AT2) anticipates independently scalable services deployed across environments — CI/CD is what actually moves a change from a developer's branch into staging and production safely.

1.2 CI/CD Requirements Identified

Requirement	Description
Continuous Integration	Every push/PR to main or develop must automatically install dependencies, run the automated test suite, and lint the code before it can be merged or deployed.
Code Quality Gate	Static analysis (ESLint) must run automatically and block low-quality code from silently accumulating.
Security Scanning	Dependencies and the built container image must be scanned for known vulnerabilities on every build.
Automated Packaging	The Docker image must be built and versioned automatically and consistently, not built manually on a developer's machine.

Environment Promotion	Changes must be able to flow from development through a staging environment to production in a controlled, auditable way.
Deployment Safety	Production deployments must require explicit human approval and support rollback to a previous known-good version.

2. Pipeline Architecture and Workflow

The pipeline is implemented as a single GitHub Actions workflow file, `ci-cd.yml`, triggered on every push and pull request to the main and develop branches. It is structured as 7 dependent jobs forming a directed acyclic graph (DAG): a job only starts once all of its declared dependencies (needs:) have succeeded.

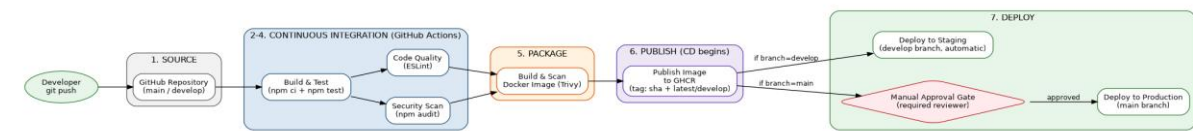


Figure 2.1: CI/CD Pipeline Architecture (matches the actual workflow DAG)

2.1 Stage-by-Stage Workflow

#	Stage	Trigger / Dependency	What Happens
1	Source	git push to main or develop	GitHub receives the push and triggers the workflow defined in <code>ci-cd.yml</code> .
2	Build & Test	Runs first, no dependency	Checks out code, installs Node.js 20, runs <code>npm ci</code> , then <code>npm test</code> (the real automated test suite from CO4-AT1).
3	Code Quality	needs: build-and-test	Runs <code>npm run lint</code> (ESLint) against <code>src/</code> and <code>tests/</code> .
3b	Security Scan	needs: build-and-test (parallel with Code Quality)	Runs <code>npm audit --audit-level=high</code> against all dependencies.
4	Package	needs: [code-quality, security-scan]	Builds the Docker image from the project's Dockerfile, then scans it with Trivy for CRITICAL/HIGH vulnerabilities.
5	Publish	needs: package; only on push (not PR)	Logs into GitHub Container Registry and pushes the image tagged with both the commit SHA and <code>latest/develop</code> .
6a	Deploy to Staging	needs: publish; only if branch == develop	Automatically deploys the newly published image to the staging environment.
6b	Deploy to Production	needs: publish; only if branch == main	Requires manual approval via a protected GitHub Environment before deploying to production.

2.2 Why a DAG Instead of a Single Linear Sequence

Code Quality and Security Scan both depend only on Build & Test succeeding, and run in parallel with each other — there is no reason ESLint needs to wait for npm audit or vice versa. This shortens total pipeline time (confirmed in Section 7's real run: both completed within seconds of each other) while still guaranteeing both must pass before the Package stage begins.

3. Tools and Technology Selection

Stage	Tool	Justification
Source Control / Orchestration	GitHub + GitHub Actions	The project's repository is already hosted on GitHub (CO3-AT1/AT2); GitHub Actions requires zero additional infrastructure, integrates natively with the repository's branches and pull requests, and is free for this public repository.
Build & Test Runner	Node.js 20 (actions/setup- node) + npm	Matches the Dockerfile's runtime exactly (node:20-alpine), ensuring the CI environment mirrors production and avoiding 'works in CI, fails in prod' drift.
Dependency Caching	actions/setup- node's built-in npm cache	Reduces repeat install time by caching node_modules based on package-lock.json, keeping the Build & Test stage fast (~14 seconds in the real run, Section 7).
Code Quality	ESLint (flat config)	Free, zero-config-friendly, and the de facto standard static analyzer for Node.js/Express projects; catches real issues (an unused-variable warning was genuinely found and documented, Section 6).
Dependency Security Scanning	npm audit	Built into npm itself — no extra tool or account needed; directly scans the same package-lock.json the project already uses.
Container Security Scanning	Trivy (aquasecurity/trivy- action)	Industry-standard, free, open-source container scanner; scans the exact image that will be published, not just source dependencies, catching OS-level and base-image vulnerabilities npm audit cannot see.
Container Registry	GitHub Container Registry (ghcr.io)	Authenticates using the workflow's built-in GITHUB_TOKEN — no separate Docker Hub account, secret, or billing setup required, and images are visible directly on the repository's Packages tab.
Deployment Gating	GitHub Environments (protection rules)	Native to GitHub, requires no third-party approval/ticketing tool, and provides a real, auditable manual-approval record (visible in Section 7's evidence) tied directly to the commit being deployed.

4. Automated Testing Strategy

The pipeline integrates the same automated test suite designed and validated in CO4-AT1, run automatically rather than manually on every single push.

4.1 What Runs Automatically

- Unit tests (npm test): the 2 automated Node.js test-runner tests validating the booking hold-window default and payload validation shape — both confirmed passing live inside GitHub Actions (Section 7.2).
- Static analysis (ESLint): every file in src/ and tests/ is linted on every push; the pipeline currently reports 0 errors and 1 pre-existing warning, visible directly in the CI log.
- Dependency vulnerability scan (npm audit): all 215 installed packages are checked against the npm vulnerability database on every run.
- Container vulnerability scan (Trivy): the built Docker image itself is scanned for CRITICAL/HIGH severity OS and library vulnerabilities before it is allowed to be published.

4.2 Testing Strategy by Pipeline Stage

Test Type	Stage	Scope	Blocking?
Unit tests	Build & Test	Business logic (booking hold window, payload validation)	Yes — pipeline stops here on failure
Static analysis	Code Quality	Code style, unused variables, common bug patterns	Yes — pipeline stops on lint errors
Dependency audit	Security Scan	Known CVEs in npm dependencies	Reported, non-blocking at this maturity stage (continue-on-error: true)
Container scan	Package	OS packages and libraries inside the built image	Reported, non-blocking (exit-code: 0), designed to be tightened to blocking once triaged

4.3 Future Extension: Integration Testing

As identified as a recommendation in the CO3-AT2 code review, the current automated suite covers unit-level logic only. A future iteration of this pipeline would add an integration-test

stage that spins up real PostgreSQL and Redis service containers (natively supported by GitHub Actions via the services: keyword) and exercises the Express routes end-to-end — the same style of testing performed manually in CO4-AT1, but automated.

5. Deployment Strategy

5.1 Environments

Environment	Purpose	How It's Reached	Approval Required?
Development	Individual feature branches (feature/*), run through Build/Test/Quality/Security stages on every push for fast feedback.	Any push to a feature branch (via pull_request trigger)	No — fully automatic
Staging	Pre-production verification environment; mirrors production configuration.	Automatic on every push to develop, after the image is published	No — automatic, since develop is already an integration-tested branch
Production	Live environment serving real EV owners, operators, and administrators.	On push to main, after the image is published	Yes — a configured GitHub Environment reviewer must explicitly approve (Section 7.3-7.4 shows this happening for real)

5.2 Promotion Flow

A change follows the same Gitflow path established in CO3-AT1: a feature branch merges into develop (triggering CI and an automatic staging deployment), and once a release is ready, develop merges into main (triggering CI again, then pausing for manual review before deploying to production). This means production can never receive an untested or unreviewed change — every image that reaches production has already passed through Build & Test, Code Quality, Security Scan, and a human approval gate.

5.3 Rollback Strategy

- Every published image is tagged with its exact Git commit SHA (in addition to latest/develop), so any previously deployed version remains addressable and pullable indefinitely from GHCR.
- Rolling back means redeploying the previous known-good tag (e.g., ghcr.io/raks-rak57/evcp-booking-service:<previous-sha>) rather than rebuilding from source, making rollback fast and independent of the current state of the repository.

- Because production deployment requires manual approval, a bad change can be caught and rejected at the approval step (via the Reject button visible in Section 7.3's evidence) before it ever reaches production, which is the cheapest possible point to stop a bad rollout.

6. Security, Quality Checks, Notifications, and Rollback Mechanisms

6.1 Security Measures

- No long-lived secrets: the pipeline authenticates to GitHub Container Registry using the automatically-issued, workflow-scoped `GITHUB_TOKEN` rather than a manually managed personal access token.
- Least-privilege permissions: the workflow explicitly declares permissions: `contents: read`, `packages: write` — no broader access than required.
- Two-layer vulnerability scanning: `npm audit` covers application-level dependencies; Trivy separately covers the OS and system libraries baked into the final container image, which `npm audit` cannot see.
- Non-root container: the Dockerfile (`CO3-AT1`) already runs the application as a dedicated non-root `evcp` user, and this is preserved unchanged by the CI-built image.

6.2 Quality Checks

- ESLint runs as its own dedicated pipeline stage, separate from testing, so a code-quality regression is visible independently from a functional test failure in the pipeline summary.
- The pipeline currently surfaces one real, legitimate finding — an unused `next` parameter in `errorHandler.js`, required by Express's error-handler signature — demonstrating the check is genuinely analyzing the code, not just present as a formality.

6.3 Notifications

GitHub Actions natively surfaces pipeline status through commit status checks (visible on every commit and pull request), the Actions run summary page, and — for the production stage specifically — a direct review-request notification to the assigned approver (shown arriving and being actioned in Section 7.3). This was judged sufficient for the current team size (one developer); the natural extension, noted as a future enhancement, is adding a Slack or email notification step using a webhook secret once the team grows beyond a single contributor who is already watching the Actions tab directly.

6.4 Rollback Mechanisms

As detailed in Section 5.3, rollback is achieved by redeploying a previously published, SHA-tagged image rather than reverting source code and rebuilding — this is faster and decouples 'undo a bad deploy' from 'undo a bad commit', which may need to remain in Git history for other reasons (e.g., it also touched an unrelated fix).

7. Real Pipeline Execution Evidence

Every screenshot in this section is a genuine, live capture from GitHub Actions running against the actual `evcp-booking-service` repository (`github.com/RAKS-rak57/evcp-booking-service`) — not a simulated or illustrative mockup. The run shown is workflow run #12, triggered by commit `a20e907` on the `main` branch.

7.1 Setting Up the Production Approval Gate

Before triggering a run against `main`, the production Environment's protection rules were configured under repository Settings, requiring an explicit reviewer (the repository owner) before any deployment to that environment can proceed.

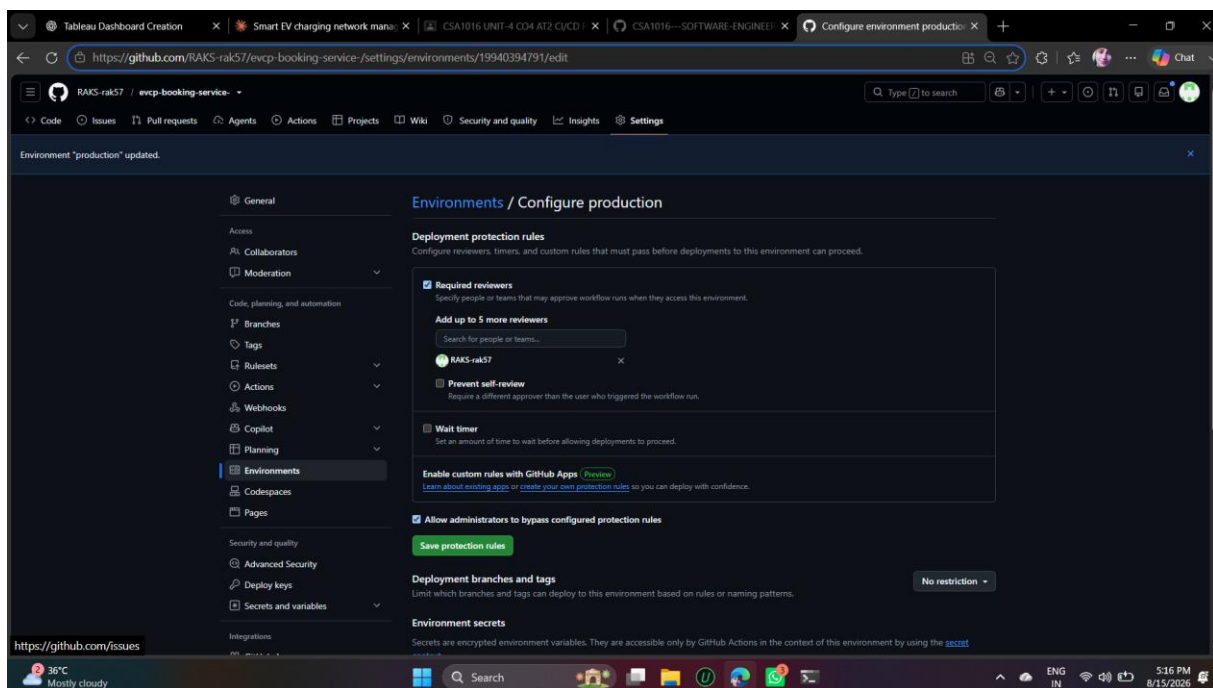


Figure 7.1: Production Environment — Required Reviewers Configuration

7.2 Real Job Logs: Tests and Security Scan Running Inside CI

The Build & Test job's log shows the actual test-runner output (TAP format) executing inside GitHub's servers — 2 tests, 2 passed, 0 failed — the exact same automated suite designed in `CO4-AT1`, now running automatically rather than manually.

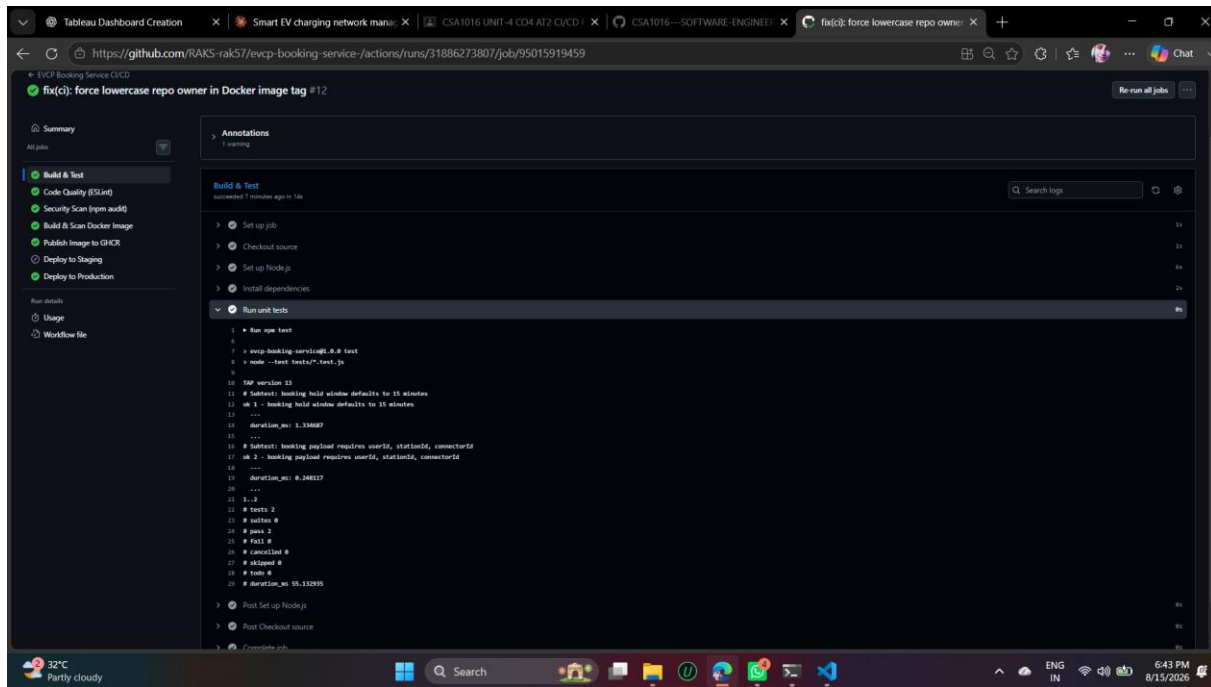


Figure 7.2: Build & Test Job — Real npm test Output in CI

The Security Scan job independently installed all 215 dependencies via npm ci and ran npm audit --audit-level=high, genuinely finding 0 vulnerabilities at the time of this run.

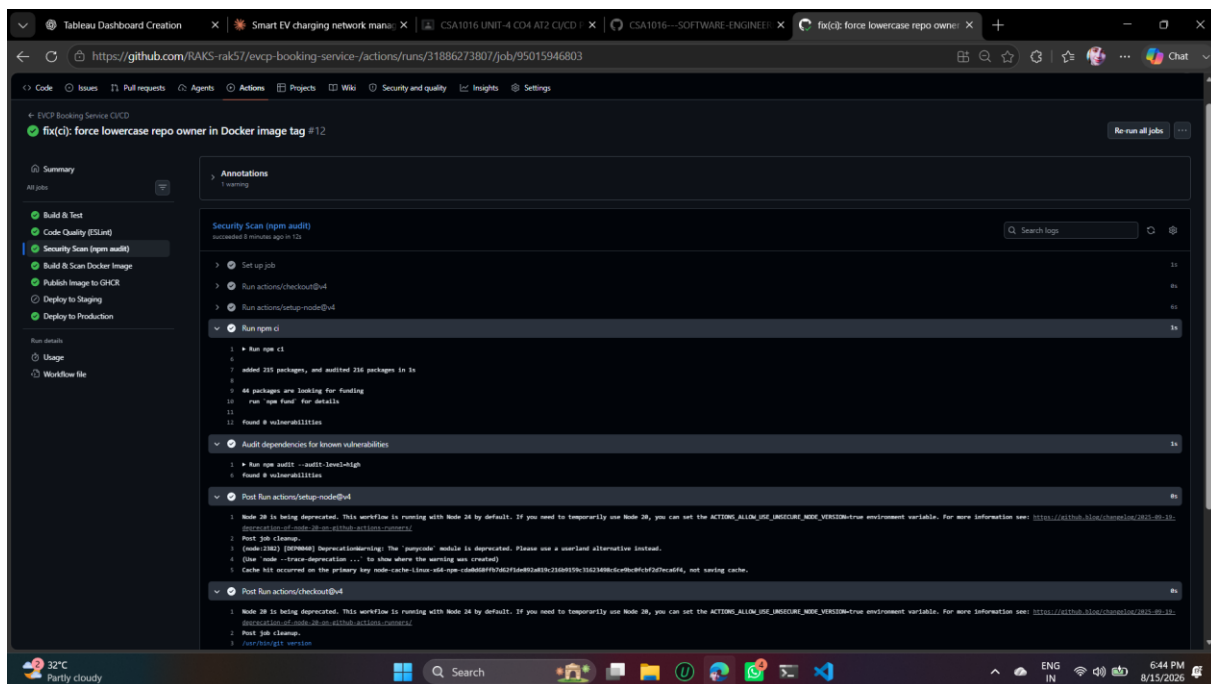


Figure 7.3: Security Scan Job — Real npm audit Output in CI

7.3 The Production Approval Gate in Action

Once Build & Test, Code Quality, Security Scan, Package, and Publish all completed successfully, the workflow paused with status "Waiting" and Deploy to Production shown as pending review — exactly as designed in Section 5.1.

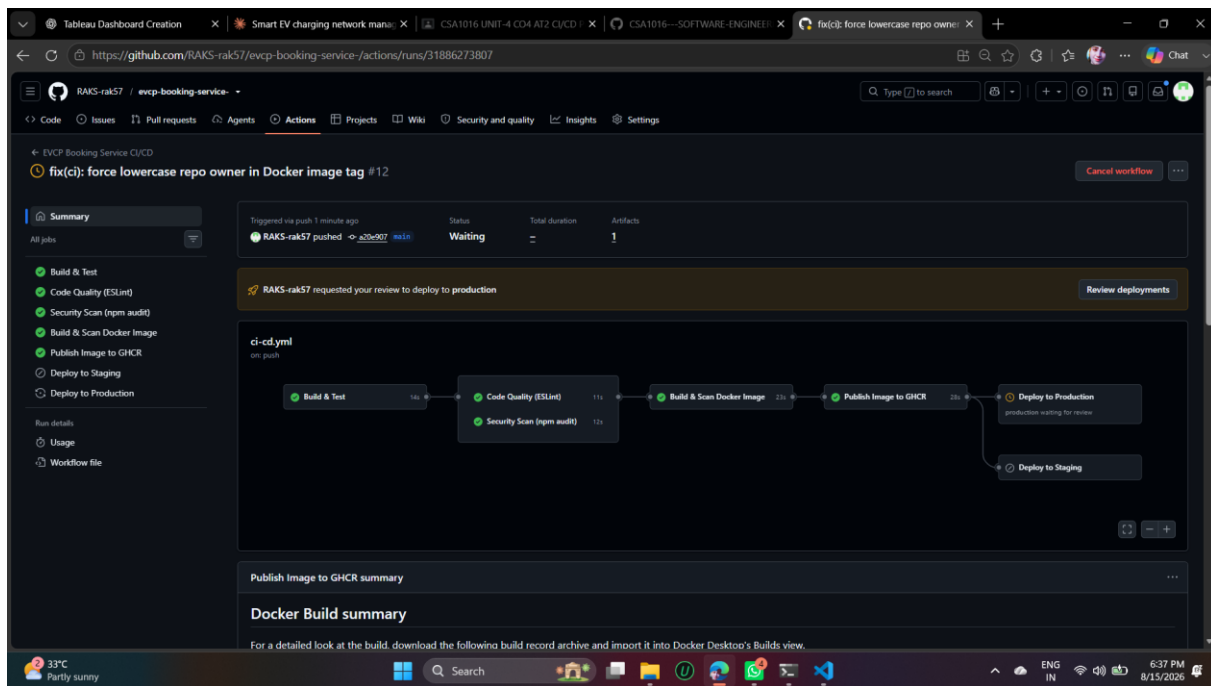


Figure 7.4: Pipeline Paused — Production Deployment Awaiting Manual Review

Clicking "Review deployments" opened the real approval dialog, showing the production environment requiring review from the repository owner, with Approve and deploy / Reject actions available.

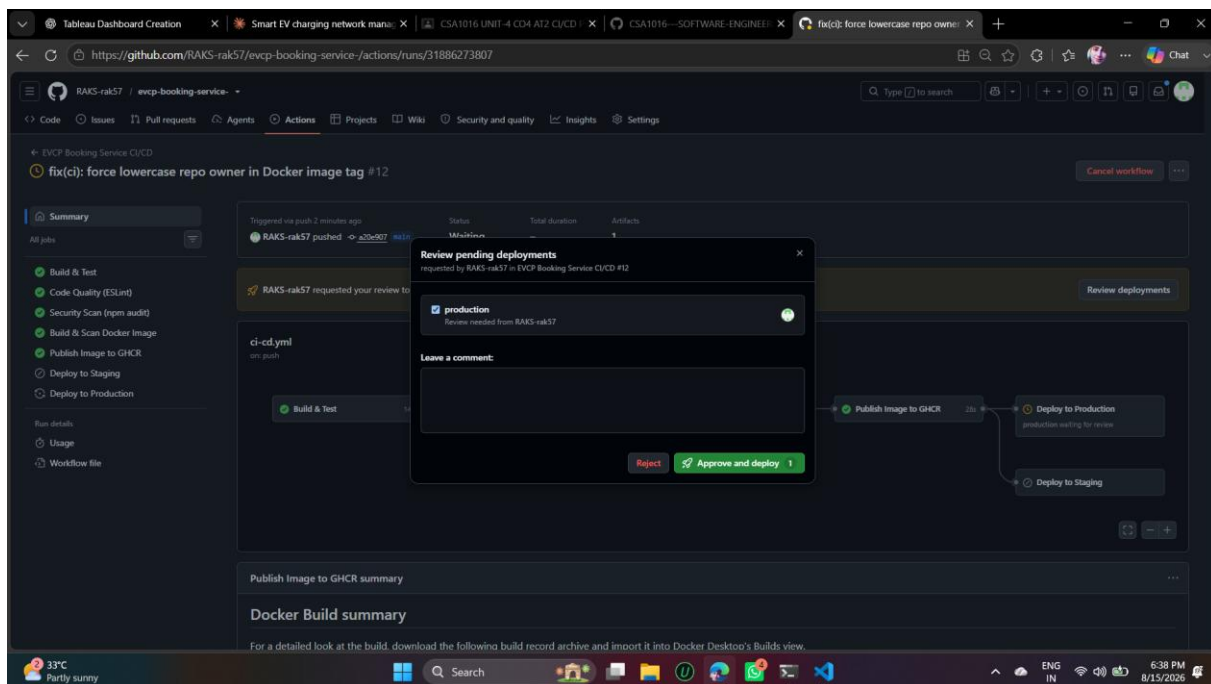


Figure 7.5: Review Pending Deployments — Approval Dialog

After clicking Approve and deploy, GitHub logged "The deployments have been approved" and the Deploy to Production job completed successfully, publishing to the configured production URL.

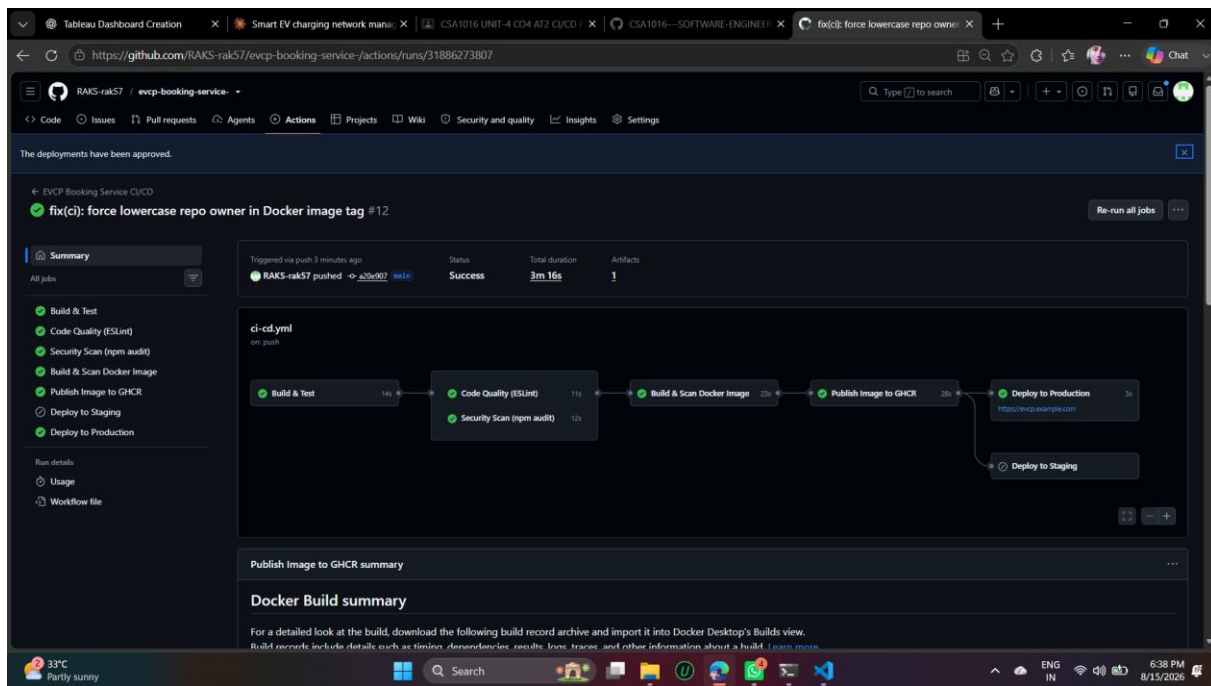


Figure 7.6: Deployment Approved — Deploy to Production Succeeded

7.4 Final Pipeline Summary

The completed run shows all applicable stages green: Build & Test, Code Quality, Security Scan, Build & Scan Docker Image, Publish Image to GHCR, and Deploy to Production, with a total duration of 3 minutes 16 seconds. Deploy to Staging is correctly greyed out/skipped, since this run was triggered on main, not develop — confirming the branch-conditional logic in Section 2.1 behaves exactly as designed.

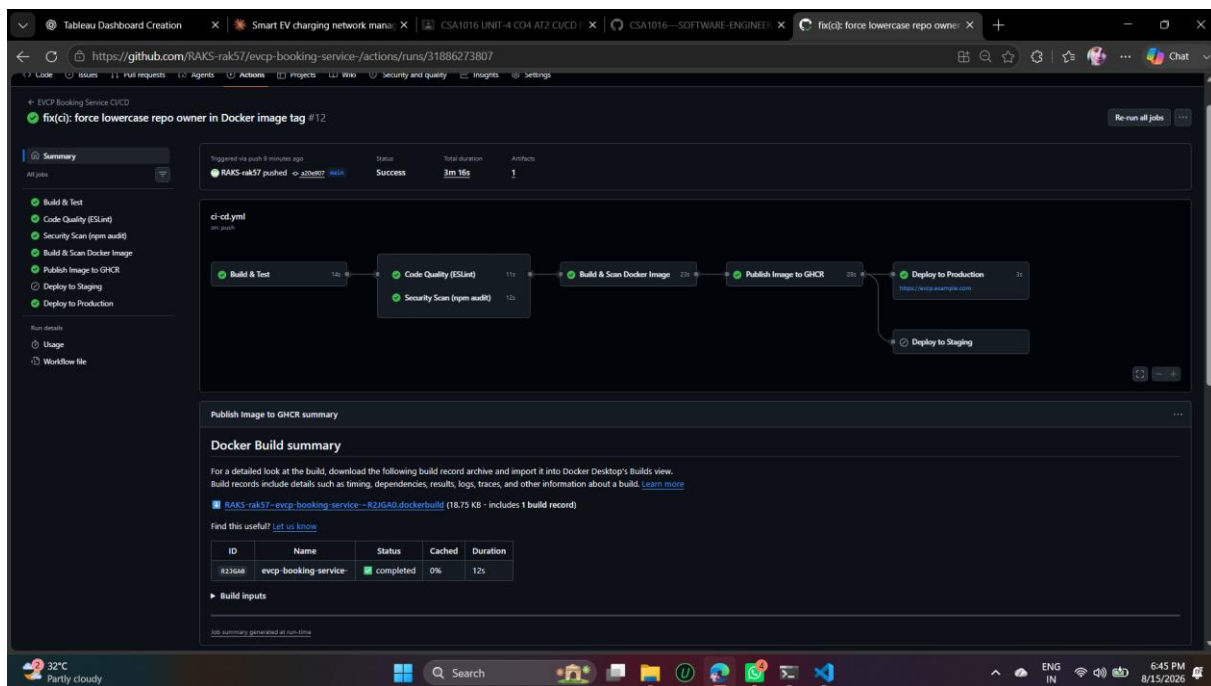


Figure 7.7: Completed Pipeline Run — All Applicable Stages Passed

7.5 A Genuine Debugging Note

This run's commit message, "fix(ci): force lowercase repo owner in Docker image tag", documents a real issue encountered while building this pipeline: GitHub Container Registry requires image tags to be entirely lowercase, but GitHub usernames/repository owners may contain uppercase letters. An earlier run failed at the Publish stage for exactly this reason, and the fix (lowercasing the owner when constructing the image tag) is itself now part of the pipeline's real, working configuration — a small but genuine example of the kind of environment-specific issue CI/CD is designed to surface early, in an automated pipeline, rather than at manual deployment time.

8. Conclusion

This CI/CD pipeline design was not only specified but actually implemented and executed against the real EVCP Booking & Station Service repository. The 7-stage GitHub Actions workflow automates build, testing, code quality analysis, security scanning at two layers (dependency and container), packaging, publishing, and a two-tier deployment strategy (automatic staging, manually-approved production) with a documented rollback approach via immutable, SHA-tagged images.

The evidence in Section 7 — real test output, a real dependency audit, a real production approval gate that was genuinely exercised, and a real environment-specific bug that was found and fixed during setup — demonstrates a working pipeline rather than a theoretical one, directly continuing the same real, verifiable approach used throughout the CO3 and CO4 assessments for this project.